

Informe de Seguridad — SIGAI-SES v1.0.0

1. Resumen Ejecutivo

El Sistema Integral de Gestion de Activos e Inventario (SIGAI-SES) implementa medidas de seguridad en multiples capas para proteger la confidencialidad, integridad y disponibilidad de la informacion de Securitas Colombia. Este informe detalla las medidas implementadas, la evaluacion OWASP Top 10, las vulnerabilidades identificadas durante la auditoria del 14 de Junio de 2026 y las recomendaciones para mejora continua.

2. Controles de Acceso

2.1 Autenticacion

Mecanismo	Implementacion
Protocolo	OAuth2 Password Flow
Formato Token	JWT (JSON Web Token) firmado con HS256
Access Token	Valido por 8 horas (configurable via ACCESS_TOKEN_EXPIRE_MINUTES)
Refresh Token	Valido por 7 dias, permite renovacion sin credenciales
Almacenamiento	Los tokens se almacenan en localStorage del navegador
Hash de Contraseñas	bcrypt con salt generado automaticamente (passlib 1.7.4)
Sesiones	Tabla <code>sesiones_usuario</code> para revocacion manual y auditoria
Rate Limiting	SlowAPI (10 requests/min en endpoint de login)
Renovacion	Interceptor Axios renueva automaticamente tokens expirados

2.2 Autorizacion (RBAC)

El sistema implementa Control de Acceso Basado en Roles con 3 niveles:

Rol	Permisos
ADMIN	Acceso total: CRUD usuarios, auditoria, configuracion global, todos los modulos
TECNICO	Consulta de inventario, registro de garantias, entregas, desmontes, reportes
TECNICO_LABORATORIO	Gestion de laboratorio, triaje de desmontes, consulta de inventario, garantias

2.3 Aislamiento por Regional

Los datos estan protegidos por codigo de regional. Un usuario de "Bogota" no visualiza equipos de "Medellin" a menos que tenga permisos de administrador nacional. Este aislamiento se implementa a nivel de consultas SQL (WHERE id_regional = ...).

3. Seguridad en la Comunicacion

3.1 Transporte

Aspecto	Estado
HTTPS	Obligatorio en produccion (certificados SSL/TLS con Let's Encrypt)
CORS	Restringido a origenes configurados en CORS_ALLOWED_ORIGINS
HSTS	Planificado para v1.1.0 (header Strict-Transport-Security)

3.2 API

Aspecto	Implementacion
Validacion de Entrada	Pydantic schemas con tipos estrictos y validacion automatica
Sanitizacion	SQLAlchemy ORM previene inyeccion SQL mediante query parameters
Serializacion	FastAPI serializa automaticamente, previniendo XSS por inyeccion HTML
Headers de Seguridad	Pendiente: CSP, X-Frame-Options, X-Content-Type-Options (v1.1.0)
Rate Limiting	Implementado en login (SlowAPI, 10 req/min). Pendiente: general (v1.1.0)
Bloqueo por intentos	Pendiente: bloqueo tras N intentos fallidos (v1.1.0)

4. Seguridad de Datos

4.1 Datos en Reposo

Tipo de Dato	Proteccion
Contraseñas	Hash bcrypt (PBKDF2-based) con salt automatico
Tokens JWT	Almacenados hasheados (SHA-256) en tabla <code>sesiones_usuario</code>
Credenciales de Equipos	Campo <code>credenciales_tecnicas</code> en tabla <code>activos</code> (pendiente de encriptacion AES-256)
Firmas Digitales	Almacenadas como texto base64 en tabla <code>actas_entrega</code>
Datos Personales	Almacenados en texto plano (identificacion necesaria para operacion)
Logs de auditoria	Valores anterior/nuevo en formato JSON, inmutables

4.2 Datos en Transito

- Encriptacion TLS/SSL en produccion
- Tokens JWT enviados en header `Authorization: Bearer <token>`
- Contraseñas solo se envian en endpoint de login, sobre HTTPS
- No se transmiten contraseñas en texto plano despues de la autentificacion

5. Auditoria y Trazabilidad

5.1 Registro de Auditoria

Todas las operaciones CRUD y eventos de login quedan registrados en la tabla `audit_logs`:

Accion	Datos Registrados
CREATE	Usuario, tabla, timestamp, valor_nuevo
UPDATE	Usuario, tabla, timestamp, valor_anterior, valor_nuevo
DELETE	Usuario, tabla, timestamp, valor_anterior
LOGIN	Usuario, tabla="usuarios", accion="LOGIN", timestamp

5.2 Monitoreo Disponible

- Endpoint `GET /health` para verificacion de estado del servidor
- Endpoint `GET /health/db` para verificacion de conexion a BD
- Endpoint `GET /metrics` con metricas del sistema (CPU, RAM via psutil)
- Logs del servidor Uvicorn con rotacion (app.log, error.log, access.log)
- Python JSON Logger para logging estructurado

6. Evaluacion OWASP Top 10

#	Categoria	Estado	Observaciones
A01	Broken Access Control	Mitigado	RBAC implementado, validacion por regional, dependencia <code>require_roles()</code>
A02	Cryptographic Failures	Mitigado parcialmente	bcrypt para contraseñas, HTTPS en produccion. Pendiente: encriptar credenciales tecnicas
A03	Injection	Mitigado	SQLAlchemy ORM previene inyeccion SQL (parameters binding). Validacion Pydantic para strings. FastAPI sanitiza salida automaticamente
A04	Insecure Design	Mitigado	Arquitectura por capas, validacion de datos en schemas Pydantic, transacciones atomicas
A05	Security Misconfiguration	Mitigado parcialmente	CORS configurado desde env, DEBUG=False en produccion. Pendiente: security headers HTTP
A06	Vulnerable Components	Mitigado	Dependencias actualizadas con versiones fijas (requirements.txt, package.json). Sin componentes vulnerables conocidos
A07	Identification Failures	Mitigado	JWT con expiracion (8h), refresh tokens (7d), sesiones revocables, bcrypt para passwords

#	Categoria	Estado	Observaciones
A08	Software Integrity Failures	Mitigado	CI/CD planificado con GitHub Actions, code review via PRs, versiones fijas de dependencias
A09	Logging Failures	Mitigado	Auditoria completa de todas las acciones CRUD, logs de servidor con rotacion
A10	SSRF	No aplica	El sistema no realiza peticiones a URLs externas no controladas

7. Hallazgos de Auditoria (14 Junio 2026)

7.1 Hallazgos Detectados y Corregidos

ID	Severidad	Descripcion	Accion	Estado
H-01	Alta	Secretos en repositorio: <code>.env</code> con <code>SECRET_KEY</code> y <code>ADMIN_PASSWORD</code> en texto plano en 3 ubicaciones	Archivos saneados, creados <code>.env.example</code>	CORREGIDO
H-02	Alta	CORS permisivo: <code>allow_origins= ['*']</code> en <code>main.py</code>	Ahora configurable desde variable <code>CORS_ALLOWED_ORIGINS</code>	CORREGIDO
H-03	Media	Credenciales de prueba en <code>locustfile.py</code> (<code>admin123</code>)	Ahora usa variables de entorno	CORREGIDO
H-04	Baja	Debug prints: <code>console.log</code> en <code>Users.tsx</code>	Eliminados	CORREGIDO
H-05	Media	Lazy loading en relaciones async SQLAlchemy	Cambiado a <code>selectinload</code> en <code>crud_deliveries</code>	CORREGIDO

7.2 Vulnerabilidades Pendientes de Correccion (v1.1.0)

ID	Severidad	Descripcion	Mitigacion Propuesta
V-01	Media	Credenciales tecnicas de activos almacenadas en texto plano (campo <code>credenciales_tecnicas</code>)	Encriptar con AES-256 antes de almacenar
V-02	Media	Tokens JWT almacenados en <code>localStorage</code> (vulnerable a XSS)	Migrar a <code>httpOnly</code> cookies o implementar refresh token rotation
V-03	Baja	Rate limiting solo en login, no en otros endpoints	Agregar middleware <code>SlowAPI</code> a toda la API
V-04	Baja	Security headers HTTP no configurados (CSP, HSTS, X-Frame-Options)	Agregar middleware o configurar en Nginx

ID	Severidad	Descripcion	Mitigacion Propuesta
V-05	Baja	No hay bloqueo por intentos fallidos de login	Implementar bloqueo temporal tras 5 intentos fallidos
V-06	Media	Pipeline CI/CD no implementado	Añadir GitHub Actions con lint + tests + build

7.3 Riesgos Aceptados

ID	Justificacion
V-02	El equipo asume el riesgo de localStorage por simplicidad de implementacion. Mitigado parcialmente por validacion de tokens y modo DEBUG desactivado en produccion
Almacenamiento de datos personales en texto plano	Necesario para la operacion del sistema. Mitigado por control de acceso RBAC y auditoria completa

8. Proteccion de Datos Personales

8.1 Datos Personales Procesados

Tipo de Dato	Finalidad	Base Legal
Nombre, email, cedula, codigo empleado	Identificacion de usuarios del sistema	Relacion laboral / contractual
Datos de contacto de clientes (nombre, NIT, telefono, direccion)	Gestion comercial y operativa	Relacion contractual
Regional asignada	Control de acceso por ubicacion	Necesidad operativa
Credenciales de equipos (IP, usuario, password)	Operacion y mantenimiento de equipos	Necesidad del servicio
Registro de acciones (logs de auditoria)	Auditoria y trazabilidad	Cumplimiento legal

8.2 Cumplimiento Legal (Colombia - Ley 1581 de 2012)

Requisito	Estado
Aviso de Privacidad	Documentado (ver 03_AVISO_DE_PRIVACIDAD.md)
Autorizacion del Titular	Gestionada por Securitas Colombia (fuera del alcance del sistema)
Derecho de Acceso	Gestion manual via administrador del sistema
Derecho de Consulta	Solicitud via correo a soporte@ses-securitas.com.co
Derecho de Rectificacion	A traves del modulo de usuarios o via administrador

Requisito	Estado
Derecho de Actualizacion	Actualizacion directa en el sistema o via administrador
Derecho de Supresion	Soft delete implementado en tablas items, clientes, proveedores
Derecho de Revocacion	Evaluacion por oficial de proteccion de datos
Seguridad de Datos	Implementado parcialmente (ver seccion 4)

9. Recomendaciones

Corto Plazo (v1.1.0)

1. **Encriptar credenciales tecnicas:** Implementar AES-256 para el campo `credenciales_tecnicas` en tabla `activos`
2. **Security headers:** Configurar CSP, HSTS, X-Frame-Options, X-Content-Type-Options en Nginx
3. **Rate limiting general:** Extender SlowAPI a todos los endpoints
4. **Refresh token rotation:** Implementar rotacion para mitigar riesgo de localStorage
5. **Pipeline CI/CD:** Anadir GitHub Actions con lint (ruff, ESLint) + tests (pytest) + build
6. **Bloqueo por intentos fallidos:** Implementar tras 5 intentos en login

Mediano Plazo (v1.2.0)

1. **Autenticacion de dos factores (2FA):** Implementar TOTP
2. **Monitorio con Sentry:** Integracion para deteccion de errores en produccion
3. **Pruebas de penetracion:** Contratar firma externa
4. **Notificaciones push/email:** Integracion SMTP para alertas
5. **Mejora de logging:** Implementar ELK stack o similar

Largo Plazo (v2.0.0)

1. **Certificacion ISO 27001:** Iniciar proceso de certificacion
2. **Auditoria de seguridad anual:** Por firma externa especializada
3. **Modulo de gestion de consentimientos:** Para datos personales
4. **Cifrado de datos en reposo a nivel de BD:** Implementar TDE o similar

10. Conclusion

El sistema SIGAI-SES implementa controles de seguridad solidos en las areas de autenticacion (OAuth2 + JWT + bcrypt), autorizacion (RBAC con 3 roles + aislamiento por regional) y auditoria (registro completo de todas las acciones). Se realizo una auditoria tecnica el 14 de Junio de 2026 que identifico 7 hallazgos, de los cuales 5 fueron corregidos inmediatamente.

No se identificaron vulnerabilidades criticas en la evaluacion inicial. Las vulnerabilidades de severidad media y baja identificadas estan documentadas y planificadas para correccion en las versiones v1.1.0 y v1.2.0.

Estado general: Aceptable para produccion con las mitigaciones documentadas.

Documento actualizado al: Julio 2026 - v1.0